RTL Track

Lab T-RTL3: Timing-Driven RTL

Pipelining the Multiplier to a Frequency Target

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 18, 2026

Lab objective: Design `seq_mul_pipe`, a parameterized N-stage pipelined multiplier with valid/ready back-pressure that delivers one result per cycle, choose its cut points from timing reports, and pick the number of stages that meets a given frequency target at the lowest cost.

Duration	2 weeks (approximately 30–40 hours)
Prerequisites	Labs R1 (`seq_mul`, `comb_mul`), V1 (testbench and SVA), P1 and P2 (synthesis, GLS, STA), P3 (`sweep`)
Designs	`seq_mul_pipe` (new), `seq_mul` testbench (DUT selection added)
Reused in	The pipelining and back-pressure patterns are reused in any later datapath block (e.g. a CPU execute stage)

Contents

1 Overview **3**

2 The Target **3**

2.1 Latency and Throughput . . . 4

3 Pipelining Theory **4**

3.1 Cutting a Combinational Path . . . 4

3.2 Register Balancing and Retiming . . . 5

3.3 Valid Propagation . . . 5

3.4 Back-Pressure . . . 6

3.4.1 Option A: Stall All Stages . . . 6

3.4.2 Option B: Skid Buffer at the Output . . . 6

3.5 Occupancy Diagram . . . 7

4 Reading the Critical Path to Choose Cut Points **8**

5 Design Specification: `seq_mul_pipe` **9**

5.1 Parameters and Interface . . . 9

5.2 Behavior . . . 9
5.3 Baseline Micro-Architecture . . . 9
5.4 Block Diagram . . . 10
5.5 Stretch: Radix-4 Booth Recoding . . . 11

6 Verification **11**
6.1 What Changes in the V1 Environment . . . 12
6.2 New Tests . . . 12
6.3 Pipeline Assertions . . . 13

7 Implementation and the N Sweep **14**
7.1 Flow . . . 14
7.2 Sweep . . . 14

8 Optional: Automatic Retiming versus Manual Pipelining **15**

9 Requirements and Tasks **16**

10 Repository and Flow **16**

11 Deliverables **17**

12 Assessment **18**

13 Review Questions **18**

14 References **18**

1 Overview

In Lab P2 you measured the two multipliers of Lab R1. `seq_mul` runs at a high clock frequency but needs about XLEN cycles per result; `comb_mul` produces a result every cycle but its single long combinational path limits the clock to a fraction of what the rest of a core would run at. A real processor wants both: the multiplier must not lower the core's clock, and it must accept a new multiplication every cycle so that a loop of independent multiplies does not stall the pipeline. The standard answer is a *pipelined* multiplier. Ibex, for example, offers single-cycle, fast multi-cycle and slow iterative RV32M options and lets the integrator choose; CVA6 uses a pipelined multiplier with a fixed latency.

Pipelining is the first technique in which the RTL designer works *from* the timing report rather than handing RTL over and hoping: you read the critical path, decide where registers go, and accept a latency cost for a frequency and throughput gain. It also introduces the problem that every pipelined block with a handshake must solve: what to do when the consumer is not ready (back-pressure).

In this lab you design `seq_mul_pipe`: the same interface and operations as `seq_mul`, a parameter `NUM_STAGES` (N), one result per cycle, and correct back-pressure. You verify it with your latency-agnostic V1 testbench, run it through the P1/P2 flow, sweep N, and choose the N that meets the frequency target set by the coordinator. Optionally you compare manual pipelining with automatic retiming in Yosys and implement a radix-4 Booth multiplier.

Where this lab sits. It builds on Lab R1 (the design and its interface), Lab V1 (the handshake-based testbench, which is the reason this lab needs almost no new verification code), Labs P1 and P2 (the flow and the reading of timing reports) and Lab P3 (parameter sweeps). Pipeline scheduling and race-free driving are as in Lab R2; SVA syntax is as in Lab V1. Nothing of that is repeated here.

By the end of this lab you should be able to:

1. Distinguish latency and throughput, and compute both for a sequential, a combinational and a pipelined implementation.
2. Estimate the clock period of an N-stage pipeline from a combinational path delay and the register overhead, and explain why returns diminish.
3. Read a critical path and decide where to cut it; balance the stages so that no stage dominates.
4. Implement valid propagation and back-pressure with stall-all-stages or a skid buffer, and justify the choice.
5. Explain retiming and register balancing, and what limits a synthesis tool's ability to retime.
6. Verify a pipelined block with a latency-agnostic testbench and pipeline-specific assertions.
7. Choose a pipeline depth for a frequency target from a measured N vs Fmax vs area vs throughput table.

2 The Target

The coordinator sets a frequency target f_{target} for the sky130 `tt_025C_1v80` corner, in the order of 250 MHz (4 ns), and the throughput requirement: **one result per cycle** at that frequency when the consumer is always ready. Start by filling this table with your Lab P2 numbers:

Metric	`seq_mul`	`comb_mul`
Minimum clock period (ns), Fmax (MHz)		
Latency (cycles) and latency (ns)		
Results per cycle, back-to-back	$\approx 1/(\text{XLEN}+k)$	1
Results per μs at Fmax		
Meets f_{target}? Meets throughput?		

Neither design meets both. The rest of the lab closes the gap.

On the Synopsys cloud: The target is defined for sky130. With the bring-up library `class.db` (`SYN_TOOL=fc STA_TOOL=pt`, the cloud default) the absolute numbers are 3–5 times larger, so use the equivalent target the coordinator gives for that library: it sits, like 250 MHz in sky130, between the two R1 designs — above the Fmax of `comb_mul` (about 55 MHz on `class.db`) and below that of `seq_mul` (about 80 MHz, one 33-bit adder per cycle). Unless told otherwise, take $f_{target} = 66.7$ MHz (15 ns) with a 10 % margin, i.e. $T_{\min} \leq 13.6$ ns. Fill the table with `make synth sta DUT_TOP=comb_mul CLK_PERIOD=15.0` (and `DUT_TOP=seq_mul`); with SAED32 or the team PDK the coordinator publishes a target for that library instead.

2.1 Latency and Throughput

- **Latency** L: cycles from the accepting edge of a request to the edge on which its result is first valid. In nanoseconds, $L \cdot T_{clk}$.
- **Throughput**: results per cycle (or per second) in steady state. A pipeline with N stages and no stalls delivers one result per cycle regardless of N.
- **Little's law**: to sustain one result per cycle with latency L, L requests must be in flight. A producer that waits for each result before sending the next request sees throughput $1/L$, not 1. This is why the testbench driver must not wait for results (Section 6).

Pipelining improves throughput and clock frequency; it never improves latency in nanoseconds, and usually makes it slightly worse, because every added register adds its own overhead.

3 Pipelining Theory

3.1 Cutting a Combinational Path

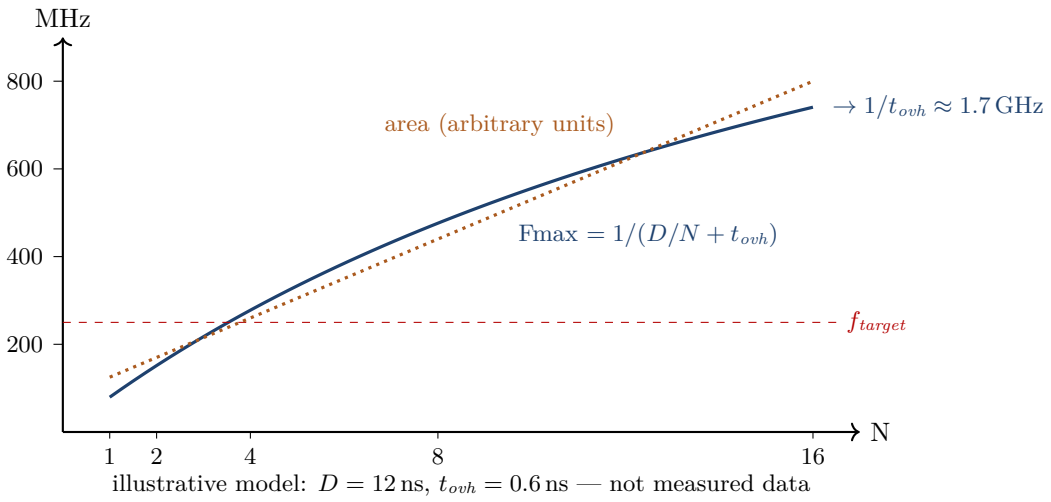
A register-to-register path must satisfy (ideal clock, as in Lab P2):

$$T_{clk} \;\geq\; t_{cq} + d_{logic} + t_{setup}$$

If a combinational block of delay D is cut into N stages of delays $d_1, \ldots, d_N$ with $\sum d_i \approx D$, the period is set by the *slowest* stage:

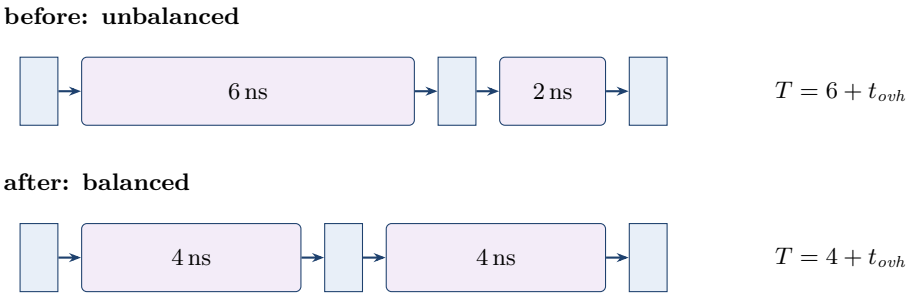
$$T_{clk}(N) \;=\; \max_i d_i + t_{ovh} \;\geq\; \frac{D}{N} + t_{ovh}, \qquad t_{ovh} = t_{cq} + t_{setup}\ (+\,t_{mux} + \text{margin})$$

The equality holds only for perfectly balanced stages. Two consequences: the frequency gain saturates at $1/t_{ovh}$, and the cost (flip-flops, and with them area and clock power) grows roughly linearly with N. Somewhere in between is the cheapest N that meets the target.



3.2 Register Balancing and Retiming

Retiming moves registers across combinational logic without changing the function seen at the block's boundary: a register at the output of a gate can be replaced by registers on all of its inputs, and vice versa. Leiserson and Saxe showed that the registers of a circuit can be repositioned optimally to minimize the clock period. *Register balancing* is the practical use: move logic from the slowest stage into its neighbors until the stage delays are similar.



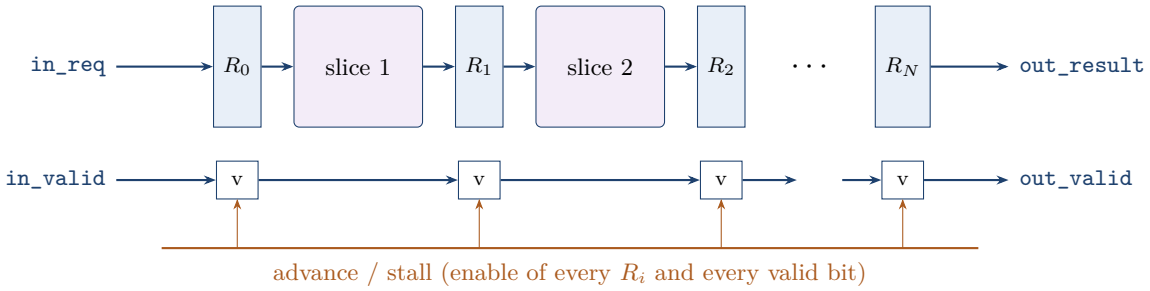
Retiming is easy on paper and constrained in practice:

- A register with an asynchronous reset or a reset value cannot be moved freely: the moved registers need reset values that reproduce the original state, which may not exist. Datapath registers without reset (allowed by Rule 6 of Lab R1) retime well; control registers do not.
- Registers with enables (a stall) must keep their enables consistent after the move.
- Moving a register changes the register names, which makes debugging, GLS comparison and constraints by name harder.
- Tools retime within a module and do not move registers across the block's ports.

Manual pipelining is retiming done by the designer: you write the registers where the balanced cut should be. Section 8 compares this with letting Yosys do it.

3.3 Valid Propagation

Each stage register holds a datapath payload (operands, partial result, operation) and a `valid` bit. Only the valid bits are reset; the payload has no reset and is loaded only when the stage advances (Lab R1, Rule 6). A stage whose valid bit is 0 holds a *bubble*: its payload is garbage and must never reach `out_result` with `out_valid` high. The valid bits form a shift register parallel to the datapath:



R_0 captures the request on the accepting edge. Slice s is the combinational logic between R_{s-1} and R_s. With N slices the latency is exactly $L = N$ cycles when nothing stalls; $N = 1$ is `comb_mul` with a handshake.

3.4 Back-Pressure

When `out_valid` is high and `out_ready` is low, the result in R_N must be held, and every item behind it must either be held as well or go somewhere. Two standard solutions:

3.4.1 Option A: Stall All Stages

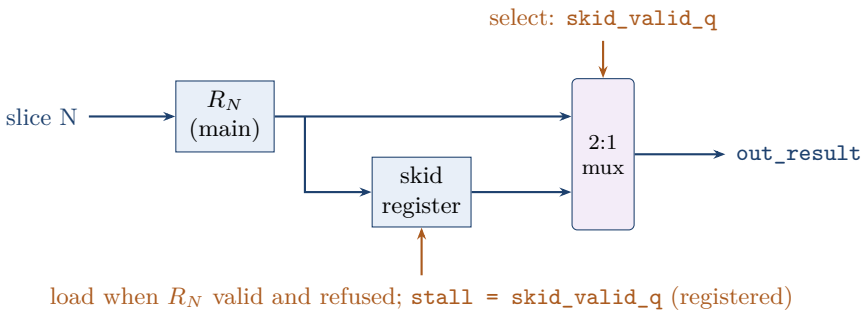
```
assign stall     = valid_q[NUM_STAGES] && !out_ready;   // global
assign in_ready  = !stall;
assign out_valid = valid_q[NUM_STAGES];
// every stage register and valid bit is enabled by !stall
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n)     valid_q <= '0;
  else if (!stall) valid_q <= {valid_q[NUM_STAGES-1:0], in_valid};
end
```

Simple, small, and correct. Its costs are timing costs:

- `out_ready` (an input port) drives the enable of every pipeline register: a high-fanout net, buffered by synthesis, on a path that starts at the block boundary. With N stages and about 4·XLEN payload bits per stage, that is hundreds of enable pins.
- `in_ready` depends combinationally on `out_ready`. Chain several such blocks and the ready path crosses all of them in one cycle — it often becomes the critical path of a system.
- Bubbles are not removed: a stalled pipeline holds its bubbles too. (A variant enables stage i when stage $i{+}1$ is empty or advancing; it collapses bubbles at the price of a ripple chain of N enables.)
- The enable adds a multiplexer (or an enable flip-flop) in front of every register, which adds to t_{ovh}.

3.4.2 Option B: Skid Buffer at the Output

A skid buffer is a second output register that catches the one item that was already moving when the consumer stopped. The pipeline is then stalled by a *registered* signal — the skid buffer being occupied — and `out_ready` drives only the skid buffer.




```
assign stall      = skid_valid_q;                        // a flip-flop output
assign in_ready   = !stall;
assign out_valid  = skid_valid_q || valid_q[NUM_STAGES];
assign out_result = skid_valid_q ? skid_data_q : res_q;

always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n)                skid_valid_q <= 1'b0;
  else if (skid_valid_q)     skid_valid_q <= !out_ready;      // drains
  else                       skid_valid_q <= valid_q[NUM_STAGES] && !out_ready;
end
always_ff @(posedge clk)                                    // payload, no reset
  if (!skid_valid_q && valid_q[NUM_STAGES] && !out_ready) skid_data_q <= res_q;
```

Walk through it: while the skid register is empty the pipeline advances every cycle. If the consumer refuses the result in R_N, that result is copied into the skid register on the same edge on which R_N takes the next item; from the next cycle the output shows the skid register and the pipeline stalls. When the consumer takes the skid entry, the stall is released and R_N is shown again. Nothing is lost, the result is stable while refused, and the latency is still N.

Property	**A: stall all stages**	**B: output skid buffer**
Flip-flops	pipeline only	+ XLEN + 1
`out_ready` → `in_ready`	combinational	none (registered)
Fanout of `out_ready`	every pipeline enable	skid register only
Output path	register	register + 2:1 mux
Stall enable on pipeline registers	yes	yes (from a flip-flop)
Throughput when `out_ready` = 1	1/cycle	1/cycle
Capacity (items held when blocked)	N + 1	N + 2

Requirement: implement one option and justify the choice in `doc/design.md` with numbers from your own STA (the path from `out_ready`, the cost in flip-flops and area). Implementing both, selected by a parameter, is a stretch goal. A third design, a free-running pipeline without enables plus an output FIFO of depth $\geq N{+}1$ with credit-based `in_ready`, removes the enable multiplexers entirely; discuss it in the review questions.

3.5 Occupancy Diagram

Requests A–F arrive back-to-back (A is accepted in the cycle before cycle 0); the consumer refuses in cycles 4 and 5 (Option A, N = 3). Each column shows the register contents during that cycle; "–" is a bubble.

Cycle	0	1	2	3	4	5	6	7	8
`in_valid/in_ready`	1/1	1/1	1/1	1/1	1/0	1/0	1/1	0/1	0/1
R_0	A	B	C	D	E	E	E	F	–
R_1	–	A	B	C	D	D	D	E	F
R_2	–	–	A	B	C	C	C	D	E
R_3 (out)	–	–	–	A	B	B	B	C	D
`out_ready`	1	1	1	1	0	0	1	1	1

Draw the same diagram for Option B in `doc/design.md` and mark the cycle in which the skid register is used.

4 Reading the Critical Path to Choose Cut Points

Do not guess where to cut: measure. Run P2 STA on `comb_mul` (or on `seq_mul_pipe` with N = 1) and look at the full path report with `-fields {slew cap input_pins}`. The *Time* column is the cumulative arrival time along the path; it tells you how much logic lies before each cell. An excerpt has this shape (illustrative):

```
Delay    Time   Description
0.000   0.000   clock clk (rise edge)
0.000   0.000 ^ stage_q[0].b[3]$_DFF_P_/CLK (sky130_fd_sc_hd__dfxtp_1)
0.352   0.352 v stage_q[0].b[3]$_DFF_P_/Q   (sky130_fd_sc_hd__dfxtp_1)
0.180   0.532 ^ _1203_/Y   (sky130_fd_sc_hd__nand2_1)     partial product
...                                                       adder array
0.410   5.917 v _2788_/COUT (sky130_fd_sc_hd__fa_1)       row 15
...                                                       final carry chain
0.288  11.402 ^ _3410_/X   (sky130_fd_sc_hd__xor2_1)      result bit 47
0.000  11.402 ^ stage_q[1].acc[47]$_DFF_P_/D
```

1. **Map the path to the RTL.** Identify the partial-product gates, the rows of the adder array and the final carry chain (Lab P1 taught you to map cells back to RTL).
2. **Find the ideal cut times.** For N stages, ideal cuts lie at arrival times $k \cdot D/N$. Find the cells at those times and the RTL structure they belong to.
3. **Cut at a structural boundary near them.** A cut in the middle of a carry chain needs registers on every partially propagated bit; a cut between rows of partial products needs only the accumulator. Prefer the latter, and accept a slightly unbalanced stage.
4. **Check the balance.** After synthesis of the pipeline, report the worst path ending in each stage register, for example `report_checks -to [get_pins stage_q[2]*/D]` (adapt the name pattern to your netlist). The spread between the slowest and the fastest stage is the imbalance; the slowest stage sets the period.
5. **Look outside the datapath.** After pipelining, the critical path is often no longer in the multiplier array: the stall/enable network, the result multiplexer, or an I/O path can take over. That is the signal that more stages will not help.
6. **Look for logic that every stage repeats.** If each slice ends in its own carry-propagate adder, cutting the array into more slices shortens only the partial-product part; the adder stays in every stage and becomes the floor of the period, and $T(N) = D/N + t_{ovh}$ no longer describes your design. The remedy is the carry-save accumulator of Section 5.

On the Synopsys cloud: PrimeTime (`STA_TOOL=pt`) writes the same information in its own format: `make sta` puts `report_timing -input_pins -nets -transition_time -capacitance` into `<top>_p<T>_setup.rpt`; the *Incr* column is the cell delay, the *Path* column the cumulative arrival time you search for the cut times $k \cdot D/N$. Fusion Compiler flattens the design and names the cells of an extracted datapath `DP_OP_*` (one per merged sum of products), so map the path through the register names at its ends (`stage_q_reg_<s>__acc__<bit>_`, `res_q_reg_<bit>_`) and the shape of the delay profile (partial-product gates, a carry-save tree of XOR/full-adder cells, then a long chain in the final adder). The per-stage check in `pt_shell` (after reading the netlist, linking and sourcing the SDC as in `flow/sta/pt_sta.tcl`): `report_timing -to [get_pins stage_q_reg_2__*/D]`, or in a loop `get_timing_paths -to $pins -max_paths 1` with `get_attribute $p arrival`; `report_timing -from [get_ports out_ready]` and `-to [get_ports in_ready]` give the back-pressure paths, `all_fanout -from [get_ports out_ready] -endpoints_only` the enable fanout.

5 Design Specification: `seq_mul_pipe`

5.1 Parameters and Interface

Parameter	Default	Description
`NUM_STAGES`	4	Number of combinational slices N (latency N cycles). Baseline: $1 \le N \le$ XLEN and N divides XLEN; checked at elaboration time
`XLEN`	from `mul_pkg`	As in Lab R1 (macro `MUL_XLEN`); `MUL_XLEN=8` must work

The ports are exactly those of `seq_mul` (Lab R1): `clk`, `rst_n`, `in_valid`, `in_ready`, `in_req` (`mul_req_t`), `out_valid`, `out_ready`, `out_result`. The supported operations are those of your R1 `seq_mul` (MUL and MULHU required; MULH and MULHSU if you implemented them in R1). The module lives in `hw/ip/seq_mul/rtl/seq_mul_pipe.sv` and uses `mul_pkg`.

5.2 Behavior

1. After reset: `in_ready = 1`, `out_valid = 0`.
2. A request is accepted on every rising edge with `in_valid && in_ready`; results leave in request order.
3. With `out_ready` held high, one request is accepted and one result is delivered every cycle, and every result appears exactly N cycles after its request was accepted.
4. `out_result` is stable while `out_valid && !out_ready`.
5. No unnecessary bubbles: `in_ready` is low only while the back-pressure mechanism is stalling (Option A: `out_valid && !out_ready`; Option B: skid register occupied).
6. No result is lost, duplicated or reordered under any pattern of `in_valid` and `out_ready`.

5.3 Baseline Micro-Architecture

The baseline is the shift-and-add algorithm of `seq_mul`, unrolled in space instead of time. `seq_mul` handles one multiplier bit per cycle with one adder; `seq_mul_pipe` gives each slice $K = \text{XLEN}/N$ multiplier bits and its own adders, and passes the partial accumulator to the next slice:

$$acc_s = acc_{s-1} + \sum_{j=(s-1)K}^{sK-1} b_j \cdot (a \ll j), \qquad acc_0 = 0$$

$N = \text{XLEN}$ is a fully pipelined bit-serial array (one adder per stage); $N = 1$ is `comb_mul`. Skeleton (unsigned operations, Option A enables):

```
localparam int K = XLEN / NUM_STAGES;          // multiplier bits per slice

typedef struct packed {
  mul_op_e           op;
  logic [XLEN-1:0]   a;
  logic [XLEN-1:0]   b;
  logic [2*XLEN-1:0] acc;
} stage_t;

stage_t stage_q [NUM_STAGES+1];                // R_0 .. R_N, payload only

always_ff @(posedge clk)                       // R_0: no reset (Rule 6)
  if (!stall) stage_q[0] <= '{op: in_req.op, a: in_req.a, b: in_req.b, acc: '0};

for (genvar s = 1; s <= NUM_STAGES; s++) begin : g_slice
  stage_t nxt;
  always_comb begin
    nxt = stage_q[s-1];
    for (int j = 0; j < K; j++)
      if (stage_q[s-1].b[(s-1)*K + j])
        nxt.acc = nxt.acc + ((2*XLEN)'(stage_q[s-1].a) << ((s-1)*K + j));
  end
  always_ff @(posedge clk)
    if (!stall) stage_q[s] <= nxt;
end
// result selection (MUL: low half, MULHU: high half) before or after R_N
```

Design decisions you must make and document:

- **Where the result is selected.** Selecting the half before R_N makes R_N narrower and the output come directly from flip-flops; selecting after adds a multiplexer to the output path.
- **Flip-flop budget.** The naive payload is about $4\,\text{XLEN} + 2$ bits per stage. Consumed bits of b need not be carried forward, and after slice s the low sK bits of acc are final. Compute your actual flip-flop count per stage and compare it with the synthesis `stat`.
- **Adder structure inside a slice.** A chain of K carry-propagate additions is simple; adding the K partial products in carry-save form and keeping acc as a sum/carry pair between slices removes the carry chain from every stage but the last, at the cost of twice the accumulator flip-flops. Try it if your slices are unbalanced by the carry chain.
- **Signed operations** (if supported): sign-extend the operands to XLEN+1 bits and treat the most significant partial product as negative, or multiply magnitudes and negate the product — the negation is a 2·XLEN-bit incrementer that needs a stage of its own. Choose and justify.

You may choose another baseline (for example `a * b` split by operand halves into four XLEN/2-bit products) if you justify it with the critical-path analysis. The interface, the behavior and the N parameter are fixed.

5.4 Block Diagram

Figure 1 shows the boxes you design, here with the output skid buffer of Option B. Three layers run in parallel: the datapath (payload registers $R_0 \dots R_N$ without reset and the combinational slices between them), the valid bits (one per register, with reset), and the control that freezes everything at once. The only signal that reaches every register is the advance enable `adv = !stall`; with Option B

it comes from a flip-flop, with Option A (dashed) from `out_ready` through one AND gate. `in_ready` is the same signal, so in Option A the ready path runs combinationally from `out_ready` to `in_ready`, and in Option B it starts at a register.

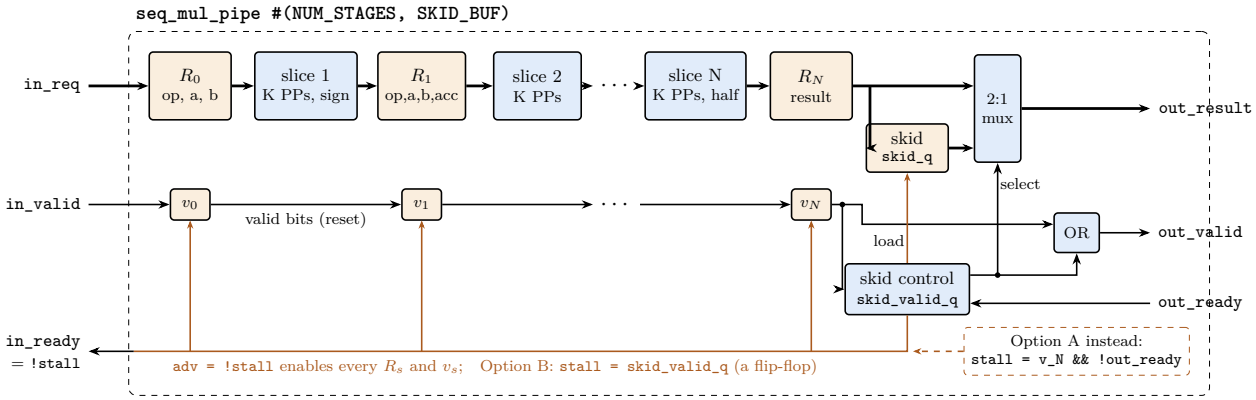


Figure 1: `seq_mul_pipe`: N slices between N+1 payload registers (orange; no reset), one valid bit per register (reset), the output skid buffer of Option B and the single advance enable. Option A replaces the skid control by the dashed combinational stall.

5.5 Stretch: Radix-4 Booth Recoding

Radix-4 (modified) Booth recoding examines overlapping triplets of the multiplier, $(b_{2i+1}, b_{2i}, b_{2i-1})$ with $b_{-1} = 0$, and selects one partial product per *two* bits, halving the number of partial products and handling two's-complement operands naturally:

$b_{2i+1}\, b_{2i}\, b_{2i-1}$	PP	$b_{2i+1}\, b_{2i}\, b_{2i-1}$	PP
0 0 0	0	1 0 0	$-2a$
0 0 1	$+a$	1 0 1	$-a$
0 1 0	$+a$	1 1 0	$-a$
0 1 1	$+2a$	1 1 1	0

Unsigned operands (MULHU) need one extra partial product (zero-extend b by two bits). Implement `seq_mul_pipe` with Booth-recoded slices, compare area and Fmax with the baseline for your chosen N, and explain the difference from the cells on the critical path.

6 Verification

The V1 testbench was required to be latency-agnostic: drivers and monitors identify transactions only by the handshake, and the scoreboard compares results in order. That requirement pays off now: the same environment verifies `seq_mul_pipe` for every N without changes to its driver, monitors, reference model or scoreboard (Figure 2). Only the DUT box changes, and a second assertion module is bound to it.

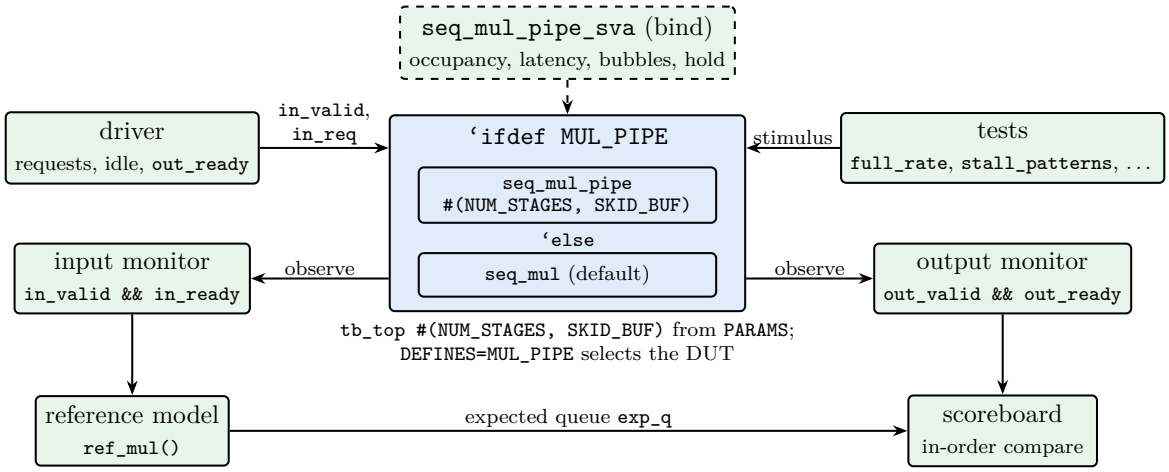


Figure 2: The V1 environment reused for `seq_mul_pipe`: every component identifies transactions only by handshakes; only the DUT instance and the bound assertion module are new.

Test term — Latency-agnostic testbench. Not a test but a property of the environment: stimulus waits only for `in_ready`, results are recognized only by `out_valid && out_ready`, and the scoreboard compares in order. A pass with it on `seq_mul` (latency XLEN) and on `seq_mul_pipe` for N = 1 . . . 32 proves the function independently of the timing; a cycle-count assumption anywhere in the checker would have broken one of them.

6.1 What Changes in the V1 Environment

- DUT selection in `hw/ip/seq_mul/dv/tb/tb_top.sv`: with `DEFINES=MUL_PIPE` the top instantiates `seq_mul_pipe #(.NUM_STAGES(NUM_STAGES))`, where `NUM_STAGES` is a parameter of `tb_top` set through `PARAMS`; the configuration print includes it. (If you implement both back-pressure options, pass their selection parameter the same way, e.g. `PARAMS="NUM_STAGES=4 SKID_BUF=0"`.) The file list of the environment must now also compile `seq_mul_pipe.sv`: point `dv/dv.f` at a file list that contains it (Section 10).
- **Driver throughput.** Check that your driver can present a new request on the cycle after acceptance. A driver that waits a cycle, or waits for the result, makes the pipeline look like `seq_mul` (Little's law). If yours does, that is a V1 testbench bug: fix it in the shared code and keep `seq_mul` passing.
- Nothing else. If anything else has to change, record why in `doc/verification.md`: it is a latency assumption that V1 should not have had.

6.2 New Tests

Test term — Back-pressure / stress test. The consumer de-asserts `out_ready` in chosen or random patterns (back-pressure), or both sides run at the maximum rate (stress). Here they are the tests that exercise the stall/skid logic: every item must survive being frozen in every stage, and the rate must return to one per cycle when the stall is released. `full_rate` is the stress test, `stall_patterns` the directed back-pressure test, `bubbles` the random one.

Test	Purpose
`full_rate`	`in_valid` and `out_ready` always 1 for 10 000 requests; measure accepted results per cycle (must be 1 after the first N cycles) and print it
`stall_patterns`	`out_ready` low for 1, 2, N, N+1 and many cycles; periodic 1-of-2 and 1-of-3 ready patterns; full pipeline stalled and released
`bubbles`	Random `in_valid` with random back-pressure; pipeline never full
existing V1 tests	Directed corner cases, random with back-pressure, exhaustive with `MUL_XLEN=8` — for every N in the sweep: `smoke` (three requests, flow alive), `corners` (0, 1, −1, most negative/positive, powers of two × every op), `back_to_back` (a new request on the cycle after each acceptance), `hold` (90 % back-pressure: held results stable), `busy_valid` (`in_valid` kept high), `random` (10 000 random requests, idle cycles and back-pressure), `exhaustive` (all $4 \cdot 2^{16}$ requests at XLEN 8)

Test term — Exhaustive test. All $4 \cdot 2^{16}$ requests of the XLEN-8 multiplier, here once per N (1, 2, 4, 8). A pass is a proof of the function for that configuration, including every carry and sign case of every slice boundary — the place where a pipelined datapath differs from `comb_mul`. With random back-pressure it also proves the ordering under stalls for those 2^{18} items.

6.3 Pipeline Assertions

Write `hw/ip/seq_mul/dv/sva/seq_mul_pipe_sva.sv` and bind it as in Lab V1:

1. **Occupancy:** a counter of accepted requests minus delivered results stays between 0 and the capacity of the chosen option (N+1 or N+2).
2. **Fixed latency without back-pressure:** if `out_ready` was high for the last N cycles, a request accepted N cycles ago produces `out_valid` now. With Option B include the cycle of acceptance itself: a result refused in that cycle fills the skid register and stalls the new request's first cycle. Count sampled edges carefully: a request seen at edge E_0 makes `out_valid` rise at E_0+N, so a concurrent assertion sees it at E_0+N+1.
3. **No unnecessary bubbles:** `!in_ready` implies the stall condition of your option.
4. **Output stability:** `out_valid && !out_ready` implies `out_valid` and a stable `out_result` in the next cycle.
5. **Valid bits never X after reset**, and (Option B) the skid register is loaded only when R_N holds a refused result.

Cover properties: pipeline full and stalled; stall released with a full pipeline; N+2 consecutive accepted requests; (Option B) skid register loaded and drained in back-to-back cycles.

Test term — Regression. All tests × seeds × configurations, one command (`make regress`), one PASS/FAIL summary. For this lab the configurations are every N of the sweep (and both back-pressure options if implemented), XLEN 32 and 8, plus the unchanged `seq_mul` configurations: a pass proves that the pipeline works for every N and that the shared environment still verifies the original design.

Test term — Negative / error-injection test. Break the design on purpose once and watch the checkers fire — for example, never load the skid register, or stall when `out_ready` is low even with an empty R_N. The scoreboard and `a_occupancy` must catch the first, `a_no_bubble` the second. Record the failing lines in `doc/verification.md`, then restore the RTL.

GLS: run the V1 testbench on the sky130 netlist (`make gls`) for N = 1 and your chosen N, with random initialization (Lab P1). Unreset payload registers are the reason to do this: a bubble that leaks garbage to `out_result` with `out_valid` high shows up here.

Test term — Gate-level simulation (GLS). The same testbench on the synthesized netlist, with random power-up values in every flip-flop. For a pipeline it proves that synthesis did not change the function (constant-register removal, merged registers, retimed registers) and that only the valid bits need a reset; the bound assertions are left out because the internal signals no longer exist.

On the Synopsys cloud: GLS with VCS on the Fusion Compiler netlist: `make gls SYN_TOOL=fc SIM=vcs DEFINES=MUL_PIPE PARAMS="NUM_STAGES=4" TEST=random` (`DEFINES=MUL_PIPE` also selects `DUT_TOP=seq_mul_pipe`). VCS starts every flip-flop at a random value (`+vcs+initreg+random`, seeded by `SEED`); `GLS_INIT=x` starts them at X instead, the four-state check that a bubble's X never reaches `out_result` while `out_valid` is high.

7 Implementation and the N Sweep

7.1 Flow

Synthesis and STA use the P1/P2 flow unchanged, with `DUT_TOP=seq_mul_pipe` and the existing `syn/seq_mul.sdc`. Two points matter for a fair sweep:

- **Measure register-to-register Fmax.** With fixed I/O delays in nanoseconds, an I/O path can dominate at short periods and hide the datapath. Report the internal paths separately: `report_checks -from [all_registers -clock_pins] -to [all_registers -data_pins]`, and report the I/O paths, in particular the one starting at `out_ready`, on their own.
- **Minimum period.** In our flow the netlist does not depend on `CLK_PERIOD` (ABC maps without a delay target), so the minimum period of the register-to-register paths is `CLK_PERIOD` − WNS from a single run. Confirm this once by re-running at the computed period; if you enable a delay target in ABC, this shortcut no longer holds and you must search the period.

On the Synopsys cloud: With `SYN_TOOL=fc STA_TOOL=pt` the register-to-register report is `report_timing -from [all_registers -clock_pins] -to [all_registers -data_pins]` in `pt_shell` (same collections as OpenSTA). Fusion Compiler, unlike our Yosys/ABC flow, *is* timing-driven: it sizes and restructures the netlist for the SDC period it synthesizes with, `SYN_CLK_PERIOD` (default 10 ns), which `make sta CLK_PERIOD=...` does not change. `CLK_PERIOD` − WNS is then the minimum period *of that netlist* (the `sweep` target iterates it); a configuration whose paths are shorter than `SYN_CLK_PERIOD` was not pushed and may be faster when synthesized tighter. Second difference: our non-physical FC run optimizes with zero wire load, while PrimeTime without parasitics applies the wire-load model of `class.db` (selected by area), so PrimeTime reports long datapaths 15–25 % slower than FC believed. A netlist synthesized exactly at the target period therefore misses it in PrimeTime; over-constrain synthesis (the sweep's `SYN_CLK_PERIOD=10` for a 15 ns target) and sign off with PrimeTime: `make synth sta DUT_TOP=seq_mul_pipe PARAMS="NUM_STAGES=8" SYN_CLK_PERIOD=15.0 CLK_PERIOD=15.0` shows the effect.

7.2 Sweep

Use the P3 `sweep` target with a configuration file for the pipeline:

```
# hw/ip/seq_mul/syn/configs_pipe.txt
NUM_STAGES=1
NUM_STAGES=2
NUM_STAGES=4
NUM_STAGES=8
NUM_STAGES=16
```

```
NUM_STAGES=32
```

Fill this table (CSV from the sweep, plus the `seq_mul` and `comb_mul` rows from Lab P2) and plot Fmax and area against N:

Design	L (cyc)	FFs	Area (μm^2)	$T_{\min}$ (ns)	Fmax (MHz)	Mresults/s	L (ns)
`seq_mul`							
`comb_mul`							
N = 1	1						
N = 2	2						
N = 4	4						
N = 8	8						
N = 16, 32							

Then choose N:

1. The chosen N meets f_{target} with the margin the coordinator specifies (wires and the clock tree are absent from our STA, Lab P2; a margin of 10–20 % is typical at this stage).
2. It is the smallest such N, or you justify a larger one (for example, better balance or a shorter `out_ready` path).
3. State the cost against the next smaller N: area, flip-flops, latency in cycles and in nanoseconds.
4. Compare the fitted model $T(N) = D/N + t_{ovh}$ with your measurements. Where does it fail, and why?

For the chosen N, add a stage-balance table (worst arrival time into each stage register) and the critical path with its RTL mapping.

8 Optional: Automatic Retiming versus Manual Pipelining

Write `seq_mul_retime #(NUM_STAGES)`: `comb_mul`'s single `a * b` expression followed by N−1 extra register stages at the output, without reset on the datapath registers and with a valid shift register beside them (no back-pressure; `out_ready` is ignored). It has the same latency as `seq_mul_pipe` but all registers at the end; a retiming tool should move them into the multiplier.

1. Synthesize it with the normal flow: Fmax should equal that of `comb_mul`. Why?
2. Synthesize it with register retiming enabled in Yosys (the `-dff` option of `abc`, used by `synth -retime`; check `help abc` and `help synth` of the installed version, the options have changed between versions). Add this as an optional switch of `flow/syn/yosys_synth.tcl` (for example an environment variable `SYN_RETIME=1`), not as a copy of the script.
3. Compare Fmax, area and flip-flop count with `seq_mul_pipe` at the same N. Where did the registers end up? Did the valid register chain or a reset prevent anything from moving?
4. Run the V1 testbench on the retimed netlist (GLS) to confirm the function at the boundary is unchanged.

A poor retiming result is a valid outcome; explain it. Commercial synthesis tools retime more aggressively (for example Design Compiler's `optimize_registers`), and teams still pipeline by hand where control logic, resets and back-pressure are involved.

On the Synopsys cloud: Fusion Compiler retimes during `compile_fusion` / `compile_logical` when it is enabled on a hierarchy before the compile: `set_optimize_registers true -modules {seq_mul_retime}` (options `-justification_effort low|medium|high` for the reset-state search and `-print_critical_loop`); `set_dont_retime` protects cells or modules, and the app option `compile.retiming.optimization_priority` sets what retiming optimizes (names checked with `set_optimize_registers -help` and `report_app_options *retim*` in FC 2025.06-SP1; `compile_fusion` itself has no retime option). In our flow this is an optional switch of `flow/syn/fc_synth.tcl` (e.g. `SYN_RETIME=1` calling `set_optimize_registers` before `compile_logical`), the counterpart of the Yosys `abc -dff` switch. Design Compiler's `optimize_registers` is not available: DC is installed but not licensed on the cloud. Formality (`make lec`) needs the SVF file FC writes to verify a retimed netlist; GLS remains the simple check.

9 Requirements and Tasks

1. Fill the target table of Section 2 from your Lab P2 results.
2. Analyze the `comb_mul` critical path and write down the ideal and the chosen cut points for N = 2, 4 and 8.
3. Write the `seq_mul_pipe` section of `doc/spec.md` (parameters, behavior, latency, capacity) and of `doc/design.md` (block diagram, chosen back-pressure option and justification, occupancy diagram, flip-flop budget).
4. Implement `seq_mul_pipe`, lint-clean, following all R1 rules; datapath registers without reset, valid bits with reset.
5. Add DUT selection to the V1 testbench, the new tests and the pipeline assertions; all V1 tests pass for every N in the sweep and with `MUL_XLEN=8` exhaustively for N = 1, 2, 4, 8; `seq_mul` still passes.
6. Add the pipeline configurations to `dv/regress.list`.
7. Run GLS for N = 1 and the chosen N.
8. Run the sweep; fill the table and the plots; choose N for f_{target} and justify it; add the stage-balance table.
9. Measure the timing of the `out_ready` path for your back-pressure option and compare with the expectation of the option table.
10. Optional: the retiming experiment; Booth recoding; both back-pressure options selectable by a parameter.
11. `make check-all` still passes.

10 Repository and Flow

```
hw/ip/seq_mul/
+-- rtl/   seq_mul_pipe.sv (new), seq_mul_retime.sv (optional),
|          seq_mul_pipe.f (new: -f seq_mul.f + seq_mul_pipe.sv;
|          seq_mul.f is unchanged)
+-- dv/    dv.f (-f rtl/seq_mul_pipe.f instead of rtl/seq_mul.f),
|          tb/tb_top.sv (DUT selection), tests (full_rate, stall_patterns,
|          bubbles), sva/seq_mul_pipe_sva.sv + bind, regress.list,
|          testplan.md (pipeline features added)
+-- syn/   configs_pipe.txt (new)
+-- doc/   spec.md, design.md, verification.md, ppa.md (pipeline sections)
```

```
flow/syn/yosys_synth.tcl          optional SYN_RETIME switch
```

No new make targets. `DUT_TOP` selects the module to synthesize; the IP Makefile keeps `DUT_TOP ?= seq_mul` (a good refinement: default to `seq_mul_pipe` when `DEFINES` contains `MUL_PIPE`, so `make gls DEFINES=MUL_PIPE` cannot mix the two), and selects `RTL_FLIST = rtl/seq_mul_pipe.f` for `DUT_TOP=seq_mul_pipe`. Keeping the pipeline in its own file list leaves the meaning of `seq_mul.f` unchanged for Labs R1–P3. If the P3 sweep cannot take a different configuration file yet, add a variable `CONFIGS ?= syn/configs.txt` to `flow/mk/` (generic change).

```
make lint  IP=seq_mul
make sim   IP=seq_mul DEFINES=MUL_PIPE PARAMS="NUM_STAGES=4" TEST=full_rate
make sim   IP=seq_mul DEFINES="MUL_PIPE MUL_XLEN=8" PARAMS="NUM_STAGES=2" \
           TEST=exhaustive
make regress IP=seq_mul
make synth IP=seq_mul DUT_TOP=seq_mul_pipe PARAMS="NUM_STAGES=4"
make sta   IP=seq_mul DUT_TOP=seq_mul_pipe PARAMS="NUM_STAGES=4" CLK_PERIOD=4.0
make gls   IP=seq_mul DUT_TOP=seq_mul_pipe DEFINES=MUL_PIPE PARAMS="NUM_STAGES=4"
make sweep IP=seq_mul DUT_TOP=seq_mul_pipe CONFIGS=syn/configs_pipe.txt \
           CLK_PERIOD=4.0
```

On the Synopsys cloud: The same commands with the cloud defaults (`source env.sh`: `SIM=vcs LINT_TOOL=spyglass SYN_TOOL=fc STA_TOOL=pt`); only the periods change for `class.db`: `make lint IP=seq_mul DUT_TOP=seq_mul_pipe PARAMS="NUM_STAGES=8"` (SpyGlass `lint/lint_rtl`; add `LINT_TOOL=verilator` for the Verilator check), `make sta ... CLK_PERIOD=15.0`, `make sweep ... CLK_PERIOD=10.0` (the sweep iterates the period from there). The V1 environment needs VCS (`SIM=vcs`); `make gls` runs VCS on the Fusion Compiler netlist. PrimeTime reports land in `build/seq_mul/<cfg>/sta/`, the sweep CSV and plots in `build/seq_mul/sweep/`.

Branch `lab/trtl3-seq-mul-pipe`, tag `trtl3-done`.

11 Deliverables

The merge request contains the new RTL, the testbench changes, the assertions, the sweep configuration, the optional flow switch, and the document updates below. Every number must be reproducible with `make` from a clean checkout.

- `README.md`: new module, how to run it, results summary (chosen N, Fmax, area, latency), known limitations.
- `doc/spec.md`: `seq_mul_pipe` parameters, behavior, latency and capacity.
- `doc/design.md`: pipeline diagram; cut-point analysis with the annotated critical path; backpressure option and its justification; occupancy diagram; flip-flop budget; Booth section if done.
- `doc/verification.md`: what changed in the V1 environment and why; test and regression results for every N; assertion and cover results; measured throughput; GLS results.
- `doc/ppa.md`: the sweep table and plots, the choice of N, the stage-balance table, the `out_ready` path, the model fit, and the retiming comparison if done.

12 Assessment

Criterion	Weight
`seq_mul_pipe` RTL: correctness, parameterization, coding rules	25%
Back-pressure design and its justification with measurements	10%
Verification: V1 reuse, new tests, pipeline assertions, GLS	15%
Critical-path analysis, cut points and stage balance	15%
N sweep, table, plots and the choice of N for the target	20%
Documentation, reproducibility and review answers	15%
Total	100%

Stretch goals (retiming experiment, Booth, both options) are not weighted separately; they are discussed in the review and can compensate for minor gaps elsewhere at the coordinator's discretion.

13 Review Questions

1. Your pipeline has N = 4 and $T_{clk} = 3.6$ ns. What are its latency in nanoseconds and its throughput, and how do they compare with `comb_mul`?
2. Why does doubling N not double Fmax? Name every term of t_{ovh} in your design.
3. Where did you cut the multiplier, and why there and not at the ideal times $k \cdot D/N$?
4. Why do the payload registers have no reset, and what makes that safe? Which test or check would catch a bubble leaking out?
5. Explain the path from `out_ready` to `in_ready` in Option A. Why is it a problem in a system, even if your block meets timing alone?
6. Walk through the skid buffer cycle by cycle for a single refused result. Why is one extra register enough?
7. What would a free-running pipeline with an output FIFO and credits buy and cost compared with your option?
8. Why could the testbench verify every N without changes, and what would have broken if the V1 driver waited for each result?
9. What prevents a synthesis tool from retiming the valid bits or a register with an asynchronous reset?
10. The coordinator raises the target by 30 %. Using your table, what do you change, and what does it cost?

14 References

- C. E. Leiserson and J. B. Saxe, "Retiming Synchronous Circuitry", *Algorithmica*, 6(1), 1991.
- N. Weste and D. Harris, *CMOS VLSI Design*: chapters on sequencing overhead, pipelining and datapath multipliers (array, Booth, carry-save).
- B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*: multipliers, Booth recoding, tree and array multipliers, pipelining.

- Harris and Harris, *Digital Design and Computer Architecture: RISC-V Edition*: pipelining, latency and throughput.
- D. Gisselquist (ZipCPU), Building a Skid Buffer for AXI Processing, 2019.
- lowRISC Ibex documentation: multiplier/divider implementation options (`RV32M`); OpenHW CVA6 `multiplier.sv` — industrial multiplier pipelines.
- Yosys manual: `synth`, `abc` (`-dff`, `-D`); OpenSTA documentation: `report_checks`, `all_registers`.